Architecture 1001: x86-64 Assembly ShiftExample3Signed.c with GCC

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

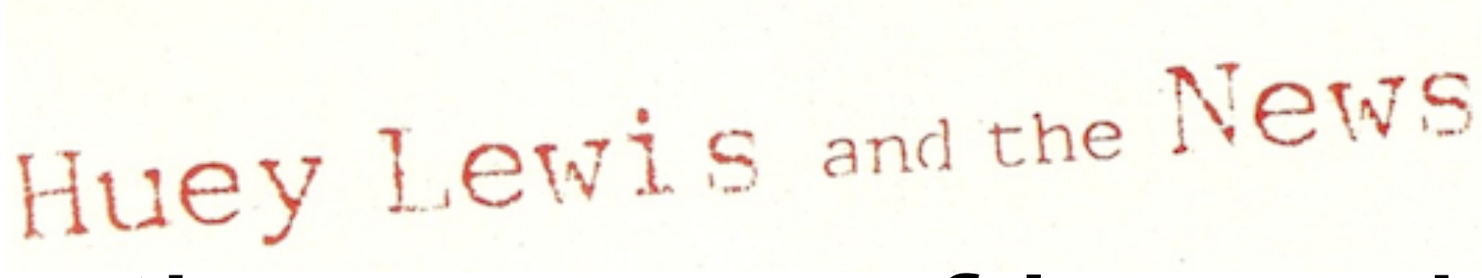
to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"



That's the power of ~~love~~ sign!

```c
#include <stdlib.h>

int main(int argc, char **argv)
{
    unsigned int a, b, c;
    a = atoi(argv[0]);
    b = a * 8;
    c = b / 16;
    return c;
}
```

```c
#include <stdlib.h>

int main(int argc, char **argv)
{
    int a, b, c;
    a = atoi(argv[0]);
    b = a * 8;
    c = b / 16;
    return c;
}
```

ShiftExample3Signed.c

Note: Compiled with /O2 “Maximize Speed”, to clear away a bit of cruft

```c
#include <stdlib.h>

int main(int argc, char **argv)
{
    int a, b, c;
    a = atoi(argv[1]);
    b = a * 8;
    c = b / 16;
    return c;
}
```

```
main:
0000000140001000  sub  rsp,28h
0000000140001004  mov  rcx,qword ptr [rdx]
0000000140001007  call qword ptr [0000000140004178h]
000000014000100D  shl  eax,3
0000000140001010  cdq
0000000140001011  and  edx,0Fh
0000000140001014  add  eax,edx
0000000140001016  sar  eax,4
0000000140001019  add  rsp,28h
000000014000101D  ret
```

```
main:
0000000140001010  sub           rsp,28h
0000000140001014  mov           rcx,qword ptr [rdx]
0000000140001017  call          qword ptr [40008368h]
000000014000101D  shl           eax,3
0000000140001020  shr           eax,4
0000000140001023  add           rsp,28h
0000000140001027  ret
```

Vs

```
main:
0000000140001010  sub           rsp,28h
0000000140001014  mov           rcx,qword ptr [rdx]
0000000140001017  call          qword ptr [40008368h]
000000014000101D  shl           eax,3
0000000140001020  cdq
0000000140001021  and           edx,0Fh
0000000140001024  add           eax,edx
0000000140001026  sar           eax,4
0000000140001029  add           rsp,28h
000000014000102D  ret
```

Changed

CDQ is added as an VS-ism. It's necessary for the math to work out, but I feel like I've only run into it *once ever* in the wild. So I don't consider it that important for beginners to know and I'm skipping it. But you can feel free to come back and read this code once we've gone through the RTFM section.

ShiftExample3Signed.c

Note: Compiled with -O1, to clear away a bit of cruft

```c
#include <stdlib.h>

int main(int argc, char **argv)
{
    int a, b, c;
    a = atoi(argv[1]);
    b = a * 8;
    c = b / 16;
    return c;
}
```

```
0x0000555555555149 <+0>:  endbr64
0x000055555555514d <+4>:  sub    $0x8,%rsp
0x0000555555555151 <+8>:  mov 0x8(%rsi),%rdi
0x0000555555555155 <+12>: mov    $0xa,%edx
0x000055555555515a <+17>: mov    $0x0,%esi
0x000055555555515f <+22>: callq  0x555555555050 <strtol@plt>
0x0000555555555164 <+27>: lea 0x0(,%rax,8),%edx
0x000055555555516b <+34>: lea 0xf(%rdx),%eax
0x000055555555516e <+37>: test   %edx,%edx
0x0000555555555170 <+39>: cmovns %edx,%eax
0x0000555555555173 <+42>: sar    $0x4,%eax
0x0000555555555176 <+45>: add    $0x8,%rsp
0x000055555555517a <+49>: retq
```

ShiftExample3Signed.c

Note: Compiled with -O1, to clear away a bit of cruft

```c
#include <stdlib.h>

int main(int argc, char **argv)
{
    int a, b, c;
    a = atoi(argv[1]);
    b = a * 8;
    c = b / 16;
    return c;
}
```

```
0x0000555555555149 <+0>:  endbr64
0x000055555555514d <+4>:  sub    $0x8,%rsp
0x0000555555555151 <+8>:  mov 0x8(%rsi),%rdi
0x0000555555555155 <+12>: mov    $0xa,%edx
0x000055555555515a <+17>: mov    $0x0,%esi
0x000055555555515f <+22>: callq  0x555555555050 <strtol@plt>
0x0000555555555164 <+27>: lea 0x0(,%rax,8),%edx
0x000055555555516b <+34>: lea 0xf(%rdx),%eax
0x000055555555516e <+37>: test   %edx,%edx
0x0000555555555170 <+39>: cmovns %edx,%eax
0x0000555555555173 <+42>: sar    $0x4,%eax
0x0000555555555176 <+45>: add    $0x8,%rsp
0x000055555555517a <+49>: retq
```

CMOVcc - Conditional MOV

- Like "Jcc", the cc stands for some conditional code like e, ne, g, le, nbe, s, ns
- If the condition is met, the MOV happens, else not

Instructions we now know (32)

- NOP
- PUSH/POP
- CALL/RET
- MOV
- ADD/SUB
- IMUL
- MOVZX/MOVSX
- LEA
- JMP/Jcc (family)
- CMP/TEST
- AND/OR/XOR/NOT
- INC/DEC
- SHR/SHL/SAR/SAL
- DIV/IDIV
- REP STOS
- REP MOVS
- LEAVE
- **CMOVcc**

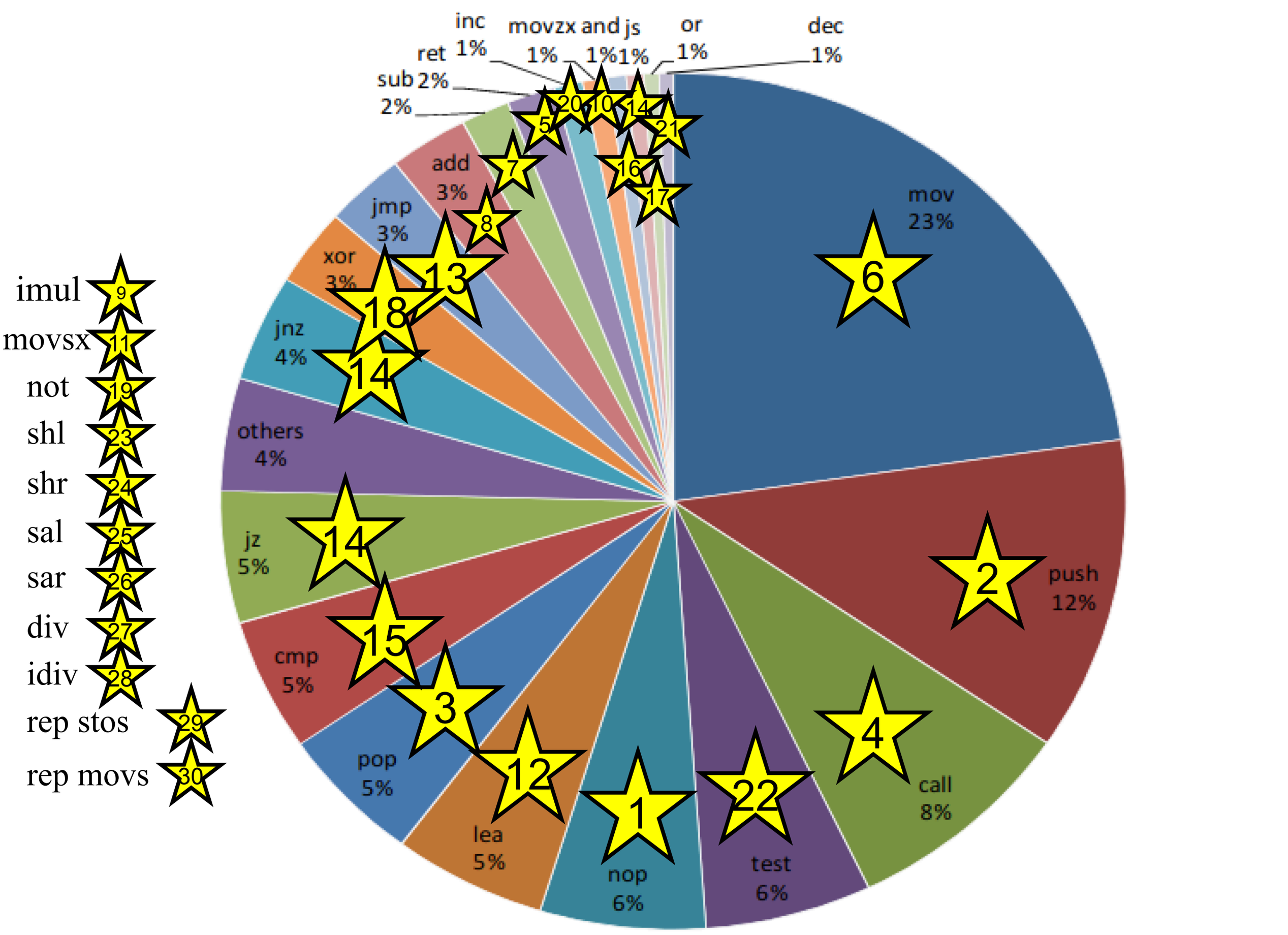


Fig. 2 Web browsers' instruction frequencies

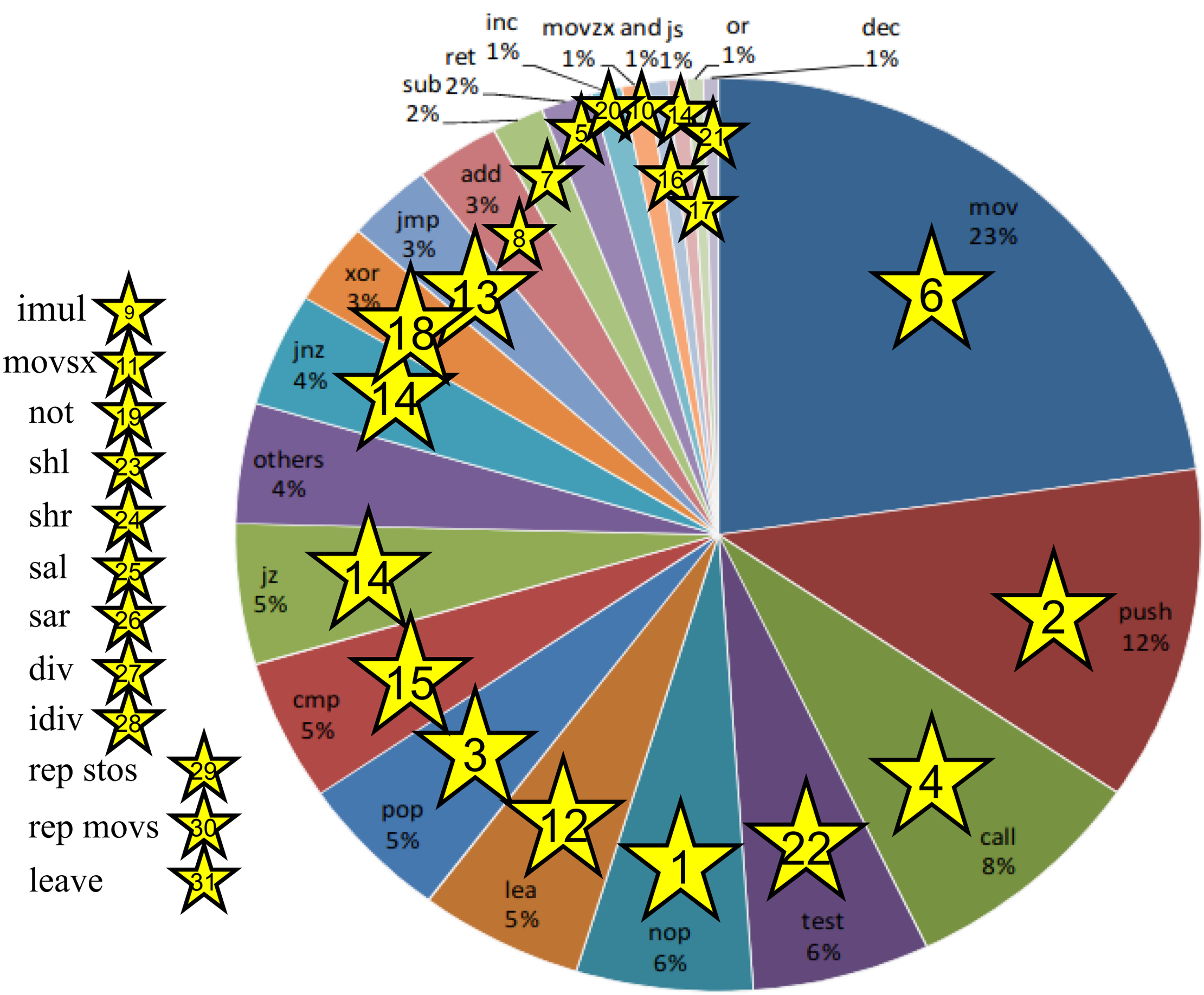


Fig. 2 Web browsers' instruction frequencies

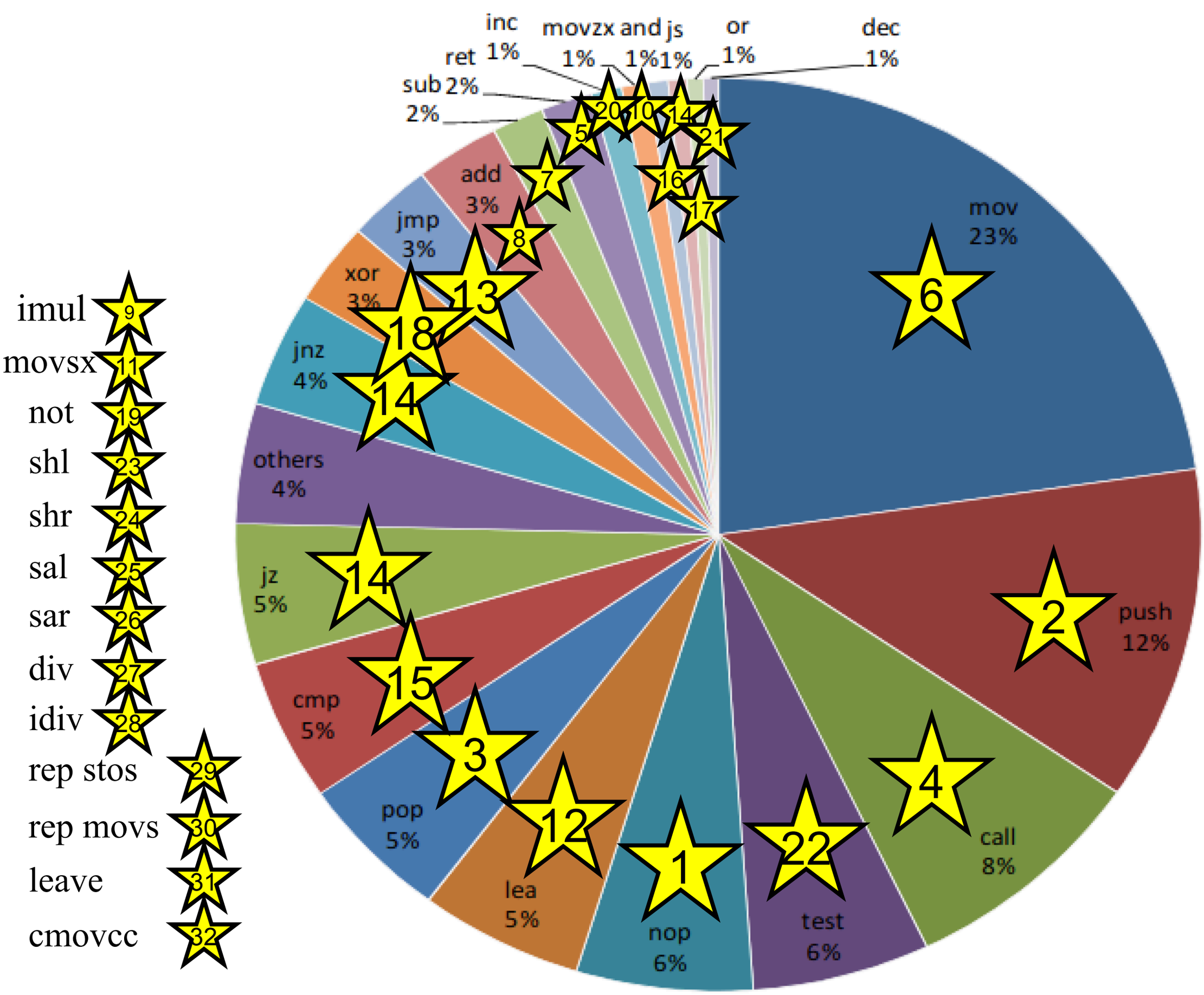


Fig. 2 Web browsers' instruction frequencies